NeuroCamo

Camouflaged Object Detection with a Spiking Neural Network

Technical walkthrough - architecture, datasets, training and results

Model	NeuroCamo v4 - spiking CNN + attention bridge + dense mask head
Parameters	329,096 (0.33 M) - backbone alone is 93 K
Input	128 x 128 RGB, Poisson-encoded to 16 binary timesteps
Datasets	COD10K + NC4K - 8,884 train / 5,238 test images
Supervision	16 x 16 downsampled object-occupancy mask (256 labels/image)
Best result	38.6% mean IoU, 33.7% IoU@50, +0.288 confidence separation
Hardware	NVIDIA RTX 3050 Ti Laptop GPU, ~105 ms per frame
Interface	Flask dashboard on 127.0.0.1:8080 with live spike spectrogram

1. What problem NeuroCamo solves

Camouflaged object detection (COD) is the task of finding a target whose colour, texture and outline have been deliberately matched to its background. It is harder than ordinary object detection for one specific reason: ordinary detectors lean on **boundary contrast**. A car has a sharp silhouette against a road. A camouflaged sniper does not - the whole point of the camouflage is to destroy that silhouette. So a detector must reason about subtle texture discontinuity and structural irregularity instead of edges.

Our second constraint is deployment. The intended platform is a battery- or solar-powered edge device (Jetson-class, or eventually neuromorphic silicon such as Intel Loihi), mounted on a drone or a static perimeter post. That rules out a 60-million-parameter transformer. We need something that is small, and whose computation is **event-driven** so that it costs energy only when something changes.

Those two constraints together are why we chose a **Spiking Neural Network (SNN)** rather than a conventional CNN, and they are the honest answer to "why not just use YOLO" - a question addressed in full in Section 9.

1.1 One-sentence summary for the panel

NeuroCamo converts an image into a train of binary spikes, processes it through a 3-layer spiking convolutional network whose neurons integrate charge over 16 timesteps, fuses the temporal spike activity into a single spatial map using an attention bridge, and predicts a 16 x 16 occupancy grid from which the bounding box is read off directly.

2. Why a spiking neural network

A conventional artificial neuron computes a weighted sum and applies a smooth activation - it outputs a real number, every layer, every frame, unconditionally. A **spiking** neuron instead behaves like a biological one: incoming current accumulates on a membrane, leaks away slowly, and the neuron emits a single binary spike only when the membrane crosses a threshold, after which it resets.

2.1 The neuron model we use (Leaky Integrate-and-Fire)

```
V(t+1) = beta * V(t) + W . x(t) - V_th * S(t) S(t) = 1 if V(t) > V_th else 0 (the spike)
```

- **beta = 0.9** is the membrane decay (leak). 90% of the charge carries over to the next timestep, 10% leaks away. We set **learn_beta=True**, so back-propagation tunes the leak per channel - fast-decaying channels become transient edge detectors, slow ones become evidence accumulators.
- **V_th** is the firing threshold, left at the snnTorch default of 1.0.
- **S(t)** is binary. This is what makes the network cheap: a spike is a 1-bit event, so the expensive multiply-accumulate of a normal CNN degenerates into an **addition**, and neurons that do not spike cost nothing at all.

2.2 The dead-neuron problem and the surrogate gradient

The threshold function is a step, and the derivative of a step is zero everywhere (and undefined at the step). Back-propagation through it would multiply every gradient by zero and nothing would ever learn - the **dead neuron problem**. The standard fix, and the one we use, is a **surrogate gradient**: the forward pass uses the hard step, but the backward pass substitutes the derivative of a smooth approximation. We use **fast_sigmoid with slope 25** from snnTorch.

Expect to be asked this. It is the single most common SNN viva question, and the answer is: hard step forward, smooth surrogate backward.

2.3 Energy argument, stated honestly

The theoretical case is that binary sparse activations replace multiplies with adds and skip silent neurons entirely, giving one to two orders of magnitude lower energy on neuromorphic hardware. The measured caveat is that we are running on a **GPU**, which is a dense-matrix machine and gains nothing from sparsity - it must still simulate all 16 timesteps. So our 105 ms per frame is **slower** than an equivalent CNN would be on the same GPU. The energy benefit is real but only materialises on event-driven silicon. Claiming otherwise would be dishonest, and a sharp examiner will catch it.

2.4 Measured spike sparsity

The backbone reports sparsity as the fraction of neurons that stayed silent across all timesteps and layers. Our target was 85%; we currently measure roughly **29%**, i.e. about 71% of neurons fire at least once. We added a sparsity penalty to the loss (weight 0.05, target 0.85) but weighted it deliberately low, because starving the network of spikes while it was still failing to detect anything would have made the detection problem harder. Raising sparsity is listed as future work in Section 13, not claimed as achieved.

3. The pipeline, stage by stage

Five stages. Every tensor shape below is real - you can quote them.

Stage	Module / file	Input shape	Output shape
1. Poisson encode	PoissonEncoder utils/poisson_encoder.py	(1, 3, 128, 128)	(16, 1, 3, 128, 128)
2. Spiking CNN	SpikingCNNBackbone models/scnn_backbone.py	(16, 1, 3, 128, 128)	spikes (16, 1, 128, 16, 16) membrane (1, 128, 16, 16)
3. Attention bridge	ASABridge models/attention_bridge.py	spikes + membrane	(1, 128, 16, 16)
4. Dense head	NeuroCamoDenseHead models/attention_bridge.py	(1, 128, 16, 16)	heatmap (1, 1, 16, 16) offset + size (1, 2, 16, 16)
5. Decode	decode_mask() models/attention_bridge.py	heatmap logits	box (1, 4) + confidence (1, 1)

3.1 Stage 1 - Poisson rate encoding

The image is normalised to [0, 1] and each pixel value is treated as a **firing probability**. For each of the 16 timesteps we draw a Bernoulli sample per pixel: a pixel of intensity 0.9 fires in roughly 14 of 16 timesteps, one of intensity 0.1 fires in roughly 2. The result is a binary tensor of shape (16, 1, 3, 128, 128).

```
x = torch.clamp(x * self.gain, 0.0, 1.0) x_repeated = x.unsqueeze(0).repeat(self.timesteps, 1, 1, 1, 1) spike_train = torch.bernoulli(x_repeated) # values in {0, 1}
```

This mimics the retina, which does not transmit pixel values but rates of nerve impulses. It also injects stochastic noise, which acts as a mild regulariser. We tested whether that noise was hurting us - see Section 7.4 - and it is not.

3.2 Stage 2 - the 3-layer spiking convolutional backbone

Each timestep's binary frame passes through three convolution + batch-norm + max-pool + LIF blocks. Crucially the LIF membrane state **persists across timesteps** while the convolution weights are shared, so the same filters are applied 16 times and evidence accumulates on the membranes.

Layer	Convolution	Resolution	Role
L1	3 -> 32, 3x3	128 -> 64	edges, fine texture
L2	32 -> 64, 3x3	64 -> 32	camouflage boundary irregularity
L3	64 -> 128, 3x3	32 -> 16	high-level semantic structure

The backbone returns three things: the layer-3 spike train over all 16 timesteps, the **final** layer-3 membrane potential, and the sparsity statistic. Only 93 K parameters live here - the whole feature extractor is smaller than a single fully-connected layer of a typical ImageNet model.

3.3 Stage 3 - ASAB, the Attention-based SNN-ANN Bridge

We now have a temporal tensor (16, 1, 128, 16, 16) but the detection head needs one spatial map. The naive fix is to average over time, which weights every timestep equally. Early timesteps, however, carry almost no information: membranes start at zero and have not yet integrated enough charge to fire meaningfully. ASAB learns that weighting instead of assuming it.

1. Average the spike train over the 16 timesteps -> (1, 128, 16, 16).

2. Global-average-pool that to (1, 128) - one scalar firing rate per channel.
3. Pass it through Linear(128 -> 32) -> ReLU -> Linear(32 -> 128) -> Sigmoid to get a per-channel attention weight in [0, 1].
4. Multiply the averaged spikes by those weights, so informative channels are amplified and noisy ones suppressed.
5. Concatenate the result with the final **membrane potential** -> (1, 256, 16, 16). Spikes are binary and lossy; the membrane is the analog sub-threshold evidence that never reached the firing threshold. A faint camouflaged edge may raise a membrane without ever triggering a spike, so discarding it loses exactly the weak signal camouflage detection depends on.
6. Fuse with a 1x1 convolution (256 -> 128) + BatchNorm + SiLU.

The name means it bridges the spiking (SNN) domain and the conventional analog (ANN) domain. It comes from the base paper, Ahmed et al. 2024.

3.4 Stage 4 - the dense mask head

Two 3x3 convolutions (128 -> 96 -> 96, each with BatchNorm + SiLU) followed by three parallel 1x1 convolution heads that keep the 16 x 16 resolution:

- **heatmap** (1 channel) - raw logits, per-cell "the object covers this cell".
- **offset** (2 channels) - sub-cell centre refinement, unused in mask mode.
- **size** (2 channels) - per-cell width/height, unused in mask mode.

The heatmap bias is initialised to **-4.595**, which is logit(0.01). Every cell therefore starts at probability 0.01. Without this the network begins by predicting "object everywhere", and since roughly 75% of cells are background the first few hundred steps are spent unlearning that instead of learning anything.

3.5 Stage 5 - decoding a box from the occupancy grid

This is the part that differs most from a standard detector, and it is worth rehearsing because it is where the design earns its keep. We do **not** regress the box. We threshold the 16 x 16 probability grid and take the bounding rectangle of the cells that survive.

```
peak = p.max() # per image thr = min(0.10, peak * 0.5) # adaptive threshold m = p >= thr x1, x2 = first and last column where m is True y1, y2 = first and last row where m is True box = [(x1 + x2 + 1)/2/W, (y1 + y2 + 1)/2/H, (x2+1-x1)/W, (y2+1-y1)/H] conf = mean of the top-4 cell probabilities
```

- The threshold is **adaptive**: min(0.10, peak x 0.5). A confident but faint detection still yields a full-extent box instead of collapsing to one cell.
- Confidence is the mean of the **top four** cells, not the single maximum. One hot cell is noise; a real object lights up a contiguous region.
- Because the box is the mask's extent, **width and height are never regressed**, so they cannot collapse to the dataset average. Section 6 explains why that matters enormously.

4. Datasets

We train on two public camouflaged-object benchmarks combined. Both ship pixel-accurate segmentation masks, and both use the identical directory layout `<root>/<split>/{Image, GT_Object}` with .jpg images and same-stem .png masks, so no per-dataset special-casing was needed - the loader simply accepts a list of roots.

Dataset	Train images	of which background	Test images	of which background
COD10K (Fan et al. 2020)	6,000	2,960 (49.3%)	4,000	1,974 (49.4%)
NC4K (Lv et al. 2021)	2,884	0	1,238	0
Combined	**8,884**	**2,960**	**5,238**	**1,974**

4.1 Why the background images matter so much

Almost half of COD10K is **NonCAM** - real photographs of the same habitats containing no camouflaged animal at all, with an all-black mask. These are gold for us. A detector trained only on positives learns "there is always something here, find it", which produces exactly the false-positive behaviour of drawing a box on empty grass. The NonCAM frames come from the same cameras and scenes as the positives and pass through the identical preprocessing, so there is no brightness or resolution shortcut the network can exploit to separate them cheaply.

The combined pool is **5,924 positives to 2,960 negatives**, a healthy 2:1 ratio requiring no resampling or class weighting.

4.2 Why we added NC4K

COD10K alone gave 3,040 positives. NC4K contributed 2,884 more - it nearly **doubles** the positive supervision, is drop-in compatible, and is distributionally different enough (different photographers, more human-made and military-style camouflage) to improve generalisation rather than just adding redundant samples.

4.3 Labels are derived, not hand-drawn

We never annotate boxes by hand. The loader reads the segmentation mask, thresholds it at 127, and derives both supervision signals from it:

1. The **bounding box** is the extent of the white region, converted to normalised [x_centre, y_centre, width, height]. Used for evaluation only.
2. The **16 x 16 occupancy mask** is the binary mask area-averaged down to the detector grid with cv2.INTER_AREA. Area-averaging (not nearest-neighbour) means a cell half-covered by the object gets the soft value 0.5 rather than being forced to 0 or 1. This is the actual training target.
3. An empty mask returns width = height = 0, which makes the **has_object** flag 0.0 and marks the frame as a true negative. Section 5 is the story of what happened when this line was wrong.

4.4 Augmentation

Applied to the training split only, and always to image **and** mask together where geometric: horizontal flip (50%), Gaussian blur with kernel 3 or 5 (30%, simulating atmospheric haze), and brightness/contrast jitter (50%, alpha 0.8-1.2, beta -20 to +20). We deliberately do **not** use colour jitter or hue rotation: camouflage is defined by colour agreement between object and background, and shifting hue would destroy the very signal we are asking the network to detect.

5. The label-contamination bug (the strongest story you have)

Present this deliberately. Examiners reward a team that diagnosed its own failure with measurements over a team that only reports a final number.

5.1 The symptom

The earlier model (v2) drew a box in almost the same place on every image, roughly centred, and reported ~99% confidence on **everything** - including photographs with no camouflage at all. It looked like it was working until you fed it two different images and got the same answer.

5.2 The measurement that pinned it down

We wrote a diagnostic (tools/diagnose_head.py) that runs the model over hundreds of images and reports the **standard deviation** of its predictions and the **correlation** between prediction and ground truth:

Quantity	Measured (v2)	Should have been
Std-dev of predicted centre	0.005	~0.09 (the ground-truth spread)
Correlation, prediction vs truth	0.051 / -0.133 / 0.074 / 0.010	well above 0.5
Confidence on real targets	0.998	high
Confidence on empty backgrounds	0.996	low
Separation between the two	+0.003	large and positive

A near-zero standard deviation with near-zero correlation is the signature of a model that has **stopped looking at the input** and is emitting a constant. Confidence separation of 0.003 confirmed it could not distinguish a camouflaged animal from bare ground.

5.3 The root cause - one line

In the dataset loader, the function that derives a box from a mask returned a **fake non-zero box** when the mask was empty:

```
# BEFORE (wrong) if not rows.any(): return [0.5, 0.5, 0.1, 0.1] # centre, 10% of the frame #
AFTER (correct) if not rows.any(): return [0.5, 0.5, 0.0, 0.0] # null box -> has_object =
0.0
```

Downstream, presence was inferred from the box size: `has_object = float(bbox[2] > 0.01 and bbox[3] > 0.01)`. Since 0.1 > 0.01, every one of the **2,960 empty background images became a positive example** asserting "an object is present, dead centre, occupying 10% of the frame, confidence 1.0".

That is 49.3% of every training epoch teaching a constant answer. The optimal response to contradictory labels of that kind is precisely to predict the average and ignore the image - so the network was not broken, it was correctly solving the wrong problem we had accidentally given it.

5.4 What the fix revealed

Re-evaluating v2 against corrected labels gave **9.6% IoU** against a **32.5%** "always predict the average box" baseline - the old model was 3.4x worse than a constant. The previously reported figures had been inflated by the same contamination they were measuring. Corrected ground truth also showed real objects occupy about 45-49% of the frame, not the 26% the poisoned statistics suggested.

Lesson to state out loud: we could not have found this from the loss curve. The loss was falling. Only distribution statistics of the predictions exposed it.

6. Three head designs, and why we ended on the third

Fixing the labels was necessary but not sufficient. The architecture of the output head had to change twice more.

6.1 Design A - global MLP regression (v2, abandoned)

Pool the 16 x 16 feature map to a 128-vector, then Linear(128 -> 64 -> 4) + Sigmoid to emit the box. The flaw is structural: this is a **mean-seeking estimator**. When the target centre is distributed around 0.5, the loss-minimising constant output is 0.5, and if the features are not informative enough to beat that constant, gradient descent will happily sit there forever. Averaging two candidate locations produces a box between them, which is a valid compromise under L1 loss and a nonsense answer in reality.

6.2 Design B - CenterNet-style centre heatmap (v4-dense, abandoned)

Predict a 16 x 16 heatmap whose target is a Gaussian blob at the object's centre, plus per-cell offset and size maps; take the box from the argmax cell. This removes the collapse failure mode structurally, because averaging two candidate peaks gives a **lower** heatmap score, not a compromise box.

It trained, but stalled. Four epochs on 8,884 images:

Epoch	Train loss	Val loss	IoU	Mean-box baseline	Confidence sep.
1	3.5256	3.6309	25.8%	37.0%	+0.017
2	3.1542	3.3110	26.2%	37.0%	+0.014
3	3.1231	3.3399	26.3%	37.0%	+0.011

IoU below the constant-box baseline, gaining 0.5% per epoch, and confidence separation **shrinking**. The size head sat frozen at 0.21 - which we verified is exactly the optimal constant for the dataset, i.e. it had collapsed to the mean again.

6.3 Why it stalled - the diagnosis, in three eliminations

1. **Not the architecture or the gradient path.** We overfit a deliberate 24-image subset: IoU reached 76.1%, heatmap loss fell 4.466 -> 0.006, and predicted sizes [0.624, 0.579] matched ground truth [0.623, 0.586]. A model that can memorise can learn; the plumbing is correct.
2. **Not the Poisson encoding noise.** We swept the timestep count and removed the stochasticity entirely: T=16 gave 26.8%, T=32 gave 26.3%, T=64 gave 26.0%, and noise-free analog input gave 23.4%, all against a 38.5% baseline. Sampling noise is irrelevant to this failure - and notably the analog version was **worse**, so the spikes are not the handicap.
3. **It is underfitting, not overfitting.** Train 3.1231 against val 3.3399 is a gap of about 7%. An overfitting model shows a large gap; this model could not fit even the training set.

Conclusion: the bottleneck was **supervision density** against a 93 K-parameter backbone. The Gaussian-centre target labels essentially **one cell per image** as positive. 8,884 images therefore provide about 8,884 positive cell labels in total - far too weak a signal to teach spatial reasoning.

6.4 Design C - mask supervision (v4-seg, current)

The insight is that both datasets ship full pixel masks, and the box-only pipeline was **throwing them away**. If the same 16 x 16 heatmap channel is trained as a downsampled **object occupancy mask** instead of a centre dot, then all **256 cells** carry a label - a 256x increase in gradient signal per image, at **zero extra compute**, with no change to the model at all.

- Same 329 K parameters, same 16 timesteps, same runtime.
- Width and height come from the mask's extent, so they are **never regressed** and cannot collapse to the dataset mean.
- The heatmap becomes a genuine saliency map, which is also the most convincing thing to show on screen during the demo.

The honest constraint behind this choice: widening the backbone to ~581 K parameters was the textbook fix for underfitting, but it needed roughly five hours of training. Mask supervision bought a similar gain in 27 minutes. Engineering under a deadline is a legitimate answer.

7. How we trained it

7.1 The loss function

Two complementary terms on the 16 x 16 occupancy grid, plus a sparsity nudge:

```
bce = binary_cross_entropy_with_logits(logits, target) p = sigmoid(logits).flatten(1); t = target.flatten(1) inter = (p * t).sum(1) dice = 1 - (2*inter + 1) / (p.sum(1) + t.sum(1) + 1) sp = relu(0.85 - measured_sparsity) loss = 1.0*bce + 1.0*dice.mean() + 0.05*sp
```

- **BCE** is per-cell and stable, but it is dominated by background: objects average about a quarter of the grid and a third of frames are pure background, so a model predicting "all background" already scores well.
- **Soft Dice** is scale-invariant - it measures overlap as a ratio, so a small object contributes as much as a large one. This is what stops small targets from being ignored. The +1 in numerator and denominator is Laplace smoothing so an all-empty frame gives dice = 0 rather than dividing by zero.
- **Sparsity penalty** is one-sided: it only activates when sparsity falls below 0.85, and is weighted at 0.05 so it cannot dominate detection.

7.2 Optimisation settings

Setting	Value	Reason
Optimiser	AdamW, weight decay 1e-4	decoupled decay behaves better than L2-in-Adam
Learning rate	2e-3 max, OneCycleLR, pct_start 0.3	warm up 30%, then anneal; suits very short runs
Batch size	16	fits 4 GB VRAM with a 16-timestep unrolled graph
Gradient clipping	max-norm 5.0	surrogate gradients can spike
Epochs	4 (then +12 resumed)	wall-clock budget, not a principled choice
Validation	655-frame strided subset	deterministic and class-mixed

The validation subset uses a fixed **stride** rather than the first N frames, because the Test folder sorts all CAM images before all NonCAM ones - taking the first 655 would have produced an all-positive validation set that could not measure false positives at all. Striding keeps it deterministic, so epoch-to-epoch numbers are directly comparable.

7.3 Checkpoint selection

We do **not** select on validation loss. Loss can improve while detection gets worse. We select on a composite:

```
score = mean_IoU + 0.5 * confidence_separation
```

This forces a checkpoint to be good at **both** localising the object and telling object-present from object-absent frames. When resuming training we also initialise the "best so far" value from the resumed checkpoint's own score, so a bad early epoch can never overwrite a good saved model.

7.4 Measured cost

- 0.665 s per batch of 16 -> 555 batches per epoch -> about 369 s of training per epoch, plus roughly 30 s of validation.
- Roughly **6.7 minutes per epoch** on an RTX 3050 Ti Laptop GPU (4 GB).
- Inference: about **105 ms** per frame including Poisson encoding and all 16 timesteps.

The 16 timesteps are the cost driver: one forward pass is effectively 16 convolutional forward passes with shared weights.

8. Results, and their honest limits

Metric	v2 (global MLP)	v4-dense (Gaussian)	v4-seg (mask, final)
Mean IoU	9.6%	26.3%	**38.6%**
IoU@50	~2%	~7%	**33.7%**
Mean-box baseline	32.5%	37.0%	37.0%
Confidence separation	+0.003	+0.011	**+0.288**
Validation loss	n/a (different loss)	3.3399	0.9389
Parameters	221,257	329,096	329,096

8.1 What these numbers mean in plain terms

- **Mean IoU 38.6%** - on average the predicted box overlaps the true box by about 39% of their combined area. It finds the right region; the box is loose.
- **IoU@50 33.7%** - about a third of detections would count as correct under the standard PASCAL VOC criterion of 50% overlap.
- **Mean-box baseline 37.0%** - the score obtained by ignoring the image entirely and always predicting the average training box. Any honest detector must beat this. We beat it, but not yet by a wide margin, and we report it precisely because most student projects quietly omit it.
- **Confidence separation +0.288** - the gap between mean confidence on object-bearing frames and on empty frames. This rose from +0.003 to +0.288, a 96-fold improvement, and it is the metric that actually fixed the visible false-positive behaviour.

8.2 The decoder's measured ceiling

We validated the decoder independently of the network: we fed the **ground truth** 16 x 16 masks in as if they were predictions and measured the box that came out. This isolates decoding error from learning error.

Decode threshold	0.05	0.10	0.20	0.35	0.50
Mean IoU	77.4%	**77.4%**	73.5%	66.0%	60.2%
IoU@50	94.8%	**93.1%**	86.5%	75.4%	64.0%

Two conclusions. First, this swept our default threshold from 0.35 to **0.10**, an immediate free gain of 11 IoU points that cost no training. Second, **77.4% is the ceiling** of this design - even a perfect network cannot exceed it, because a 16 x 16 grid over a 128-pixel image means each cell is 8 pixels wide and the box can only ever be accurate to that quantum. Our 38.6% is therefore about half of the achievable maximum, and the remaining gap is a training-budget problem, not a design dead end.

8.3 State this limitation before you are asked

The model was trained for **4 epochs**. That is not a converged result; it is what fitted the available time. The loss was still falling and IoU still rising when we stopped. We are reporting a working system with a clear improvement path, not a finished benchmark contender - and published COD methods reach far higher numbers using 30-60 million parameters and days of training on multiple GPUs.

9. How this differs from YOLO and other detectors

Expect this question first and answer it structurally, not defensively.

Aspect	YOLO (v5/v8)	NeuroCamo
Neuron model	Analog, continuous activations, one pass	Spiking LIF, binary events, 16 timesteps with persistent membrane state
Parameters	7 M (YOLOv8s) to 68 M (YOLOv8x)	0.33 M - roughly 20x to 200x smaller
Arithmetic	Dense multiply-accumulate everywhere	Accumulate-only where a spike occurs; silent neurons cost nothing
Target hardware	GPU / TPU, dense tensor units	Neuromorphic (Loihi, SpiNNaker) or low-power edge
Objects per image	Many, with anchors and non-max suppression	One dominant region - single-target by design
Classes	80 (COCO) or user-defined	One: camouflaged / not camouflaged
Supervision	Hand-drawn boxes	Segmentation masks, downsampled to 16 x 16
Output primitive	Regressed box offsets from anchors	Occupancy grid; the box is derived from its extent
Trained on	COCO, millions of images, many GPU-days	8,884 images, 4 epochs, one laptop GPU, 27 minutes

9.1 The fair comparison, stated plainly

A YOLO model fine-tuned on COD10K would almost certainly beat our 38.6% IoU. Say this before the panel does. It has 20-200x the parameters and the benefit of ImageNet/COCO pretraining. Our contribution is not "more accurate than YOLO"; it is **a camouflage detector that runs on event-driven hardware at 0.33 M parameters**, which is a different design point on the accuracy-energy curve. If accuracy at any energy cost were the goal, an SNN would be the wrong choice and we would say so.

9.2 Versus dedicated COD architectures (SINet, SegMaR, transformers)

Published COD methods (SINet, SINet-V2, SegMaR, HitNet, and recent transformer-based models) typically use a ResNet-50 or PVT backbone, ImageNet pretraining, multi-stage search-and-identify modules, and 448x448 or larger input. They report far better numbers than we do. They also use 30-100 M parameters and are not deployable on a battery-powered sensor. We borrowed their **problem formulation** - treat COD as segmentation, not box regression - which is exactly the pivot described in Section 6.4.

9.3 Versus a plain CNN of the same size

This is the sharpest possible question and we have partial data on it: our own ablation replaced the Poisson spike input with noise-free analog input and scored **worse** (23.4% vs 26.8%). That is not a full ANN-vs-SNN comparison - the neurons were still LIF - and we have **not** trained an equivalent non-spiking CNN. If asked whether an ANN of 0.33 M parameters would do better, the correct answer is: probably slightly better in accuracy, worse in energy on neuromorphic hardware, and we have not measured it. That ablation is the first item of future work.

9.4 Versus SAM / foundation models

Segment Anything and similar models segment beautifully but are prompted - they answer "what is at this point", not "is anything hidden here" - and are 630 M+ parameters. They solve a different problem at a wholly

different deployment cost.

10. The dashboard - what every panel means

The web interface is a single-page Flask application served on `127.0.0.1:8080`. It is bound to localhost only, so it is not reachable from the network.

10.1 Left column - controls

- **Drop zone** - upload a JPG/PNG/BMP/WEBP up to 25 MB. Large images are downscaled server-side before encoding.
- **Sample frames** - two built-in images, one camouflaged and one ordinary, so the contrast can be demonstrated without hunting for files.
- **Spike bands shown** - SPARSE / ACTIVE / SATURATED toggles that filter the spectrogram colour map by firing density. Hidden bands render grey. This is a **visualisation filter only**; it does not change the model's computation.
- **Run detection** - submits the frame to /api/detect.
- **Model panel** - which checkpoint is loaded, device, epoch, validation loss, parameter count, timestep count. Check this first if results look wrong: if it says best_model_v2.pth you are looking at the old broken model.

10.2 The metric strip

Readout	What it actually is
Gated confidence	Final confidence after any scene gate. For the v4 dense head the gate is disabled, so this equals the pre-gate value.
Pre-gate confidence	The network's raw objectness - the mean of the four highest cell probabilities in the 16 x 16 grid.
Laplacian variance	Texture complexity of the input, computed with OpenCV. A classical image statistic, not a network output.
Silent neurons	Measured spike sparsity - the fraction of neurons that never fired across all 16 timesteps. Currently around 29%.
Encode + 16 steps	End-to-end latency in milliseconds, about 105 ms.
Rate > 1%	How many of the 128 layer-3 channels are meaningfully active.

*Be ready to explain that the Laplacian variance and the old scene gates are **classical computer vision, not the SNN**. They existed only because the v2 head emitted 0.998 confidence on every input, so a hand-written filter was the only thing separating a hit from a miss. Now that the dense head produces real objectness, those gates are **bypassed** - they were actively destroying correct detections in flat or bright scenes such as snow or a sky-lined target.*

10.3 Input frame / Detection panels

Left is the frame as submitted, with its pixel dimensions. Right is the annotated result: the bounding box, the verdict badge, and the 16 x 16 objectness heatmap upscaled and overlaid. Three verdict bands are used - **DETECTED** above 0.60 confidence (green), **POSSIBLE TARGET** from 0.25 to 0.60 (amber), and **NO CAMO DETECTED** below 0.25 (red, box suppressed).

The heatmap is the honest showpiece. It is **the exact tensor the box was decoded from**, not a separately computed explanation, so the overlay mathematically cannot disagree with the box. That is a stronger claim than a post-hoc saliency method such as Grad-CAM can make.

10.4 Layer-3 spike spectrogram

A 128 x 16 matrix - one row per layer-3 channel, one column per timestep - where brightness is that channel's firing rate at that timestep. It is built by averaging the spike tensor over the 16 x 16 spatial grid.

It is the direct analogue of an audio spectrogram's frequency-bin by time-frame matrix, and it is the panel that makes the network's temporal behaviour visible: you can point at the left edge and note that early timesteps are quiet because membranes have not yet charged, which is precisely the imbalance that ASAB learns to correct.

10.5 Live IP-camera feed

The dashboard can pull an MJPEG stream from a phone running an IP-camera app and run detection frame by frame via /api/detect_stream and /api/video_feed. This demonstrates that the pipeline is not restricted to still images.

11. Working of every Python file

Two folders. `neurocamo/` is the research project - model, datasets, training. `neurocamo-web/` is the demo, which imports the research code unchanged rather than duplicating it.

11.1 utils/poisson_encoder.py

Turns one static image into a spike train. The pixel intensity is clamped to [0, 1], repeated T=16 times, and each copy is passed through `torch.bernoulli`. A pixel of 0.7 therefore fires in roughly 11 of the 16 timesteps - intensity becomes firing probability, which is rate coding. `forward_sequence()` handles video, where each real frame supplies one timestep instead of repeating a single frame.

```
x = torch.clamp(x * self.gain, 0.0, 1.0)
x_repeated = x.unsqueeze(0).repeat(self.timesteps, 1, 1, 1, 1)
spike_train = torch.bernoulli(x_repeated)
```

11.2 models/scnn_backbone.py

Three convolution + LIF blocks. Each block is Conv2d -> BatchNorm -> `snn.Leaky`, and the whole stack is stepped 16 times inside a Python loop over the timestep axis while the membrane potentials persist between steps. Returns the layer-3 spike tensor (T, B, 128, 16, 16), the final membrane potential (B, 128, 16, 16), and the measured sparsity. beta=0.9 is learnable, and the surrogate gradient is `fast_sigmoid(slope=25)`. Only 93 K parameters.

11.3 models/attention_bridge.py

Three classes matter here. **ASABridge** collapses the time axis using learned channel attention and fuses the result with the membrane potential. **NeuroCamoDenseHead** is two 3x3 convolutions at width 96 followed by three 1x1 heads for heatmap, offset and size, with the heatmap bias initialised to -4.595 so every cell starts at p = 0.01. **NeuroCamoDenseModel** chains backbone -> ASAB -> head and carries the two decoders, `decode` (peak cell + offset + size) and `decode_mask` (extent of the above-threshold cells). **NeuroCamoFullModel** is the old global-MLP model, kept only so the v2 checkpoints still load.

11.4 datasets/cod10k_dataset.py

Walks the COD10K and NC4K folders, pairs each image with its ground-truth PNG mask, and returns four tensors: the resized 128 x 128 image, the bounding box in normalised xywh, the objectness label, and - when `return_mask=True` - the mask average-pooled to 16 x 16. That last tensor is the entire reason v4 works. Augmentation is horizontal flip and mild scale/crop only; no colour jitter, because hue and saturation similarity between target and background *is* the signal.

11.5 datasets/negative_dataset.py

Supplies the NonCAM negatives - real scenes with no camouflaged object - so the confidence head has something to say no to. Without these the objectness score is meaningless because every training sample would be positive.

11.6 training/train_seg.py (the run that produced our checkpoint)

Loads both datasets with masks, builds the dense model in `mask_mode=True`, and optimises BCE + soft Dice on the 16 x 16 heatmap with a one-sided sparsity penalty. AdamW at 2e-3 with OneCycleLR, batch 16, gradient clipping at norm 5. Validation uses a strided 640-frame subset for speed, and reports IoU against the **mean-box baseline** so an improvement cannot be faked. Checkpoints are kept on the composite score `iou + 0.5 * sep`, which prevents a model that guesses one big box everywhere from winning on IoU alone. `NEUROCAMO_RESUME` continues from existing weights, and when resuming the peak learning rate is cut to

30% so the OneCycle warm-up cannot destroy the solution being resumed.

11.7 training/train_dense.py

The previous attempt: same architecture, centre-Gaussian supervision, so roughly one labelled cell per image. Kept in the repository because it is the controlled comparison that proves supervision density - not architecture - was the bottleneck. Its `iou_xywh` helper is imported by train_seg.py, so the two runs are scored by identical code.

11.8 neurocamo-web/inference.py

The bridge. It adds the research folder to `sys.path`, reads the checkpoint's `head` field to decide which model class to build *before* loading weights, and holds the encoder and network in a single long-lived `ModelBundle` so the cost is paid once at start-up. `run_inference()` calls the backbone explicitly rather than `model(spikes)`, because that keeps the layer-3 spike train available for the spectrogram. It returns the annotated image plus the JSON payload: verdict, confidence, heatmap bytes, sparsity, latency, box in both pixel and normalised coordinates.

11.9 neurocamo-web/app.py

Route	Purpose
GET /	Serves the dashboard page.
GET /api/status	Checkpoint, device, epoch, val loss, parameter and timestep counts.
POST /api/detect	Accepts one uploaded frame, returns the annotated PNG and metrics.
POST /api/detect_stream	Same, for a frame pulled from a live camera stream.
GET /api/video_feed	Proxies the MJPEG camera stream.

Binds `127.0.0.1:8080` with `debug=False, threaded=True`. Uploads are capped and a 413 handler returns a clean JSON error. `sanitize_json` strips NaN and infinity so the browser never receives invalid JSON, and cache headers are set to no-store so a stale result is never shown.

*If asked about security, answer plainly: the server binds loopback only, so it is not reachable from the network, but **/api/detect has no authentication**. That is acceptable for a local demo and would need a token plus TLS before any deployment.*

11.10 Front end and diagnostics

- `static/index.html` - panel layout and markup.
- `static/styles.css` - the dark palette, shared with our sibling project so the two look like one body of work.
- `static/app.js` - uploads the frame, draws the spectrogram and heatmap onto canvases from the quantised byte arrays, and animates the metric strip. All rendering is client-side; the server sends numbers, not charts.
- `tools/diagnose_head.py` - measures whether the head has collapsed to the dataset mean: predicted-versus-ground-truth standard deviation and correlation per coordinate. This is the script that caught the v2 failure.
- `tools/probe_centre.py` - shifts a synthetic target around the frame and checks that the predicted centre follows it, i.e. that the model is translation-sensitive at all.

12. How to run the demo

1. Activate the virtual environment and start the server: `python neurocamo-web/app.py`. Open `http://127.0.0.1:8080`.
2. **Check the model panel first.** It must read best_model_v4.pth, epoch 4, 329,096 parameters. If it reads v2, an older server process is still holding port 8080 - kill it and restart. This actually happened to us during testing.
3. Run the ORDINARY SCENE sample first. Low confidence, no box. This establishes that the detector can say no, which is the claim the negatives dataset supports.
4. Run the CAMOUFLAGED sample. Point at the heatmap and say the box was decoded from exactly that tensor.
5. Open the spectrogram and note the quiet left edge - membranes charging - then explain that ASAB learns to reweight timesteps for precisely this reason.
6. Upload an unseen image, ideally one the panel supplies. Do not hide a loose box; say the grid is 16 x 16 over 128 pixels, so one over-fired cell widens the box by about 6%, and that more epochs tighten it.

Have the numbers ready and say them without being asked: 38.6% IoU against a 37.0% mean-box baseline, 33.7% IoU@50, +0.288 confidence separation, 0.33 M parameters, 4 epochs. The weakest number is the epoch count, and owning it is far better than being caught by it.

13. Limitations and future work

13.1 What we are not claiming

- **We are not state of the art.** Dedicated COD networks reach 80%+ on these datasets with 100x our parameters. Our design point is accuracy per parameter on event-driven hardware.
- **The energy argument is theoretical on our hardware.** A GPU gains nothing from spike sparsity, so our 105 ms is slower than an equivalent CNN. The saving is real only on neuromorphic silicon, and we have not measured it there.
- **Measured sparsity is ~29%, not the 85% we target.** The penalty weight of 0.05 is too gentle. This is an honest gap between goal and result.
- **Four epochs.** Validation loss was still falling when we stopped. The decoder ceiling test says 77.4% IoU is reachable with this exact architecture, so roughly half our remaining error is simply unspent training budget.
- **Single object.** decode_mask returns one box around all above-threshold cells, so two animals in one frame produce one box spanning both.

13.2 What we would do next, in priority order

1. Train to convergence - 30 to 50 epochs. The cheapest large gain available, and it needs no code change.
2. Raise the grid from 16 x 16 to 32 x 32. Cell size drops from 8 px to 4 px, which directly attacks the quantisation error that makes boxes loose.
3. Connected-component labelling on the mask instead of a single global extent, giving genuine multi-object output.
4. Raise the sparsity weight and re-measure, so the energy claim rests on our own numbers rather than the literature's.
5. Deploy to real neuromorphic hardware - Loihi or SpiNNaker - and measure joules per frame. That is the experiment that would turn our motivation into a result.

One-paragraph summary, if you only remember one thing

NeuroCamo detects camouflaged objects with a 0.33 M-parameter spiking neural network. Images become Poisson spike trains, three spiking convolution layers extract features over 16 timesteps, an attention bridge collapses time into one spatial map, and a dense head predicts a 16 x 16 occupancy mask from whose extent the bounding box is read. Training the heatmap as a downsampled mask instead of a centre point raised IoU from 9.6% to 38.6% and confidence separation from +0.003 to +0.288, because it gave 256 labelled cells per image instead of one. Every number quoted is measured against a mean-box baseline on a held-out test split.